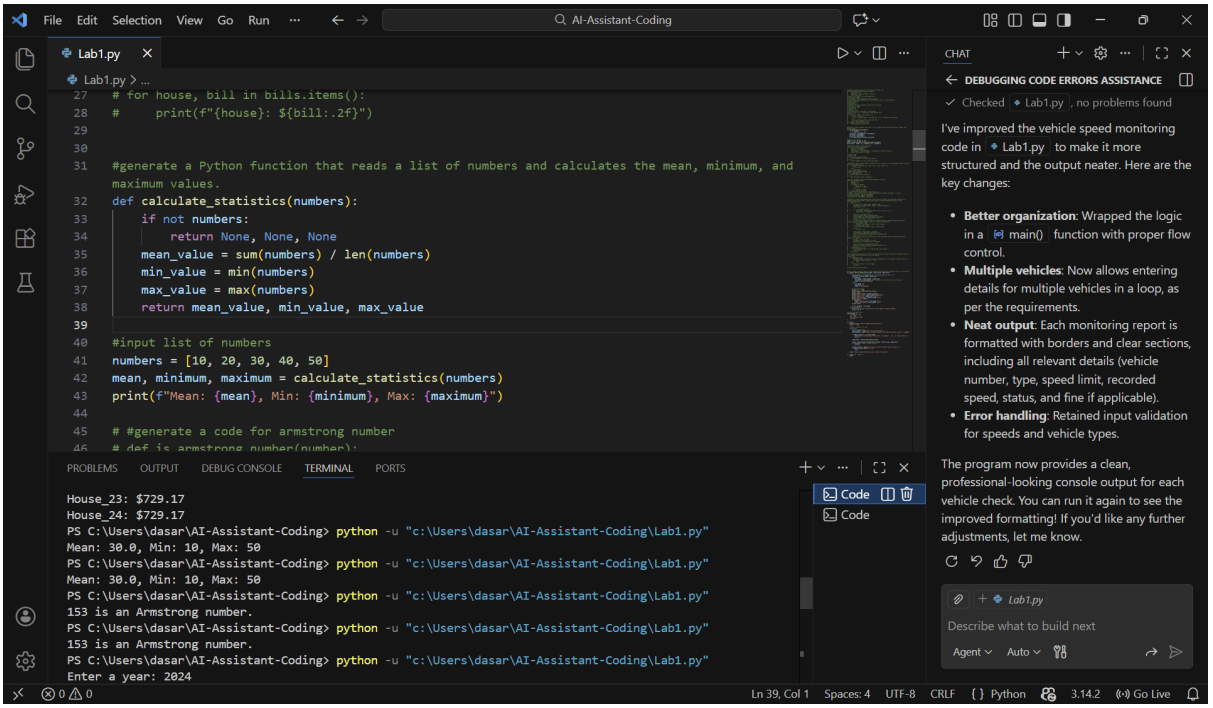
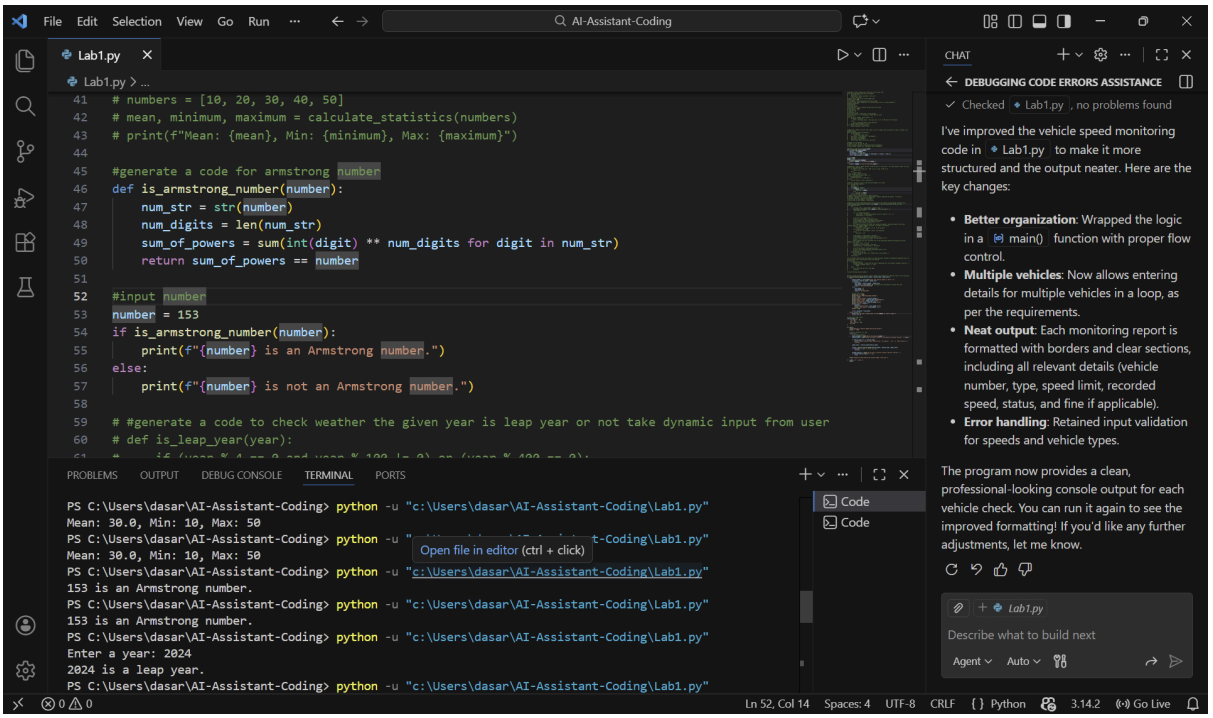
AI Assistant Coding - LAB - 1
2303A51236

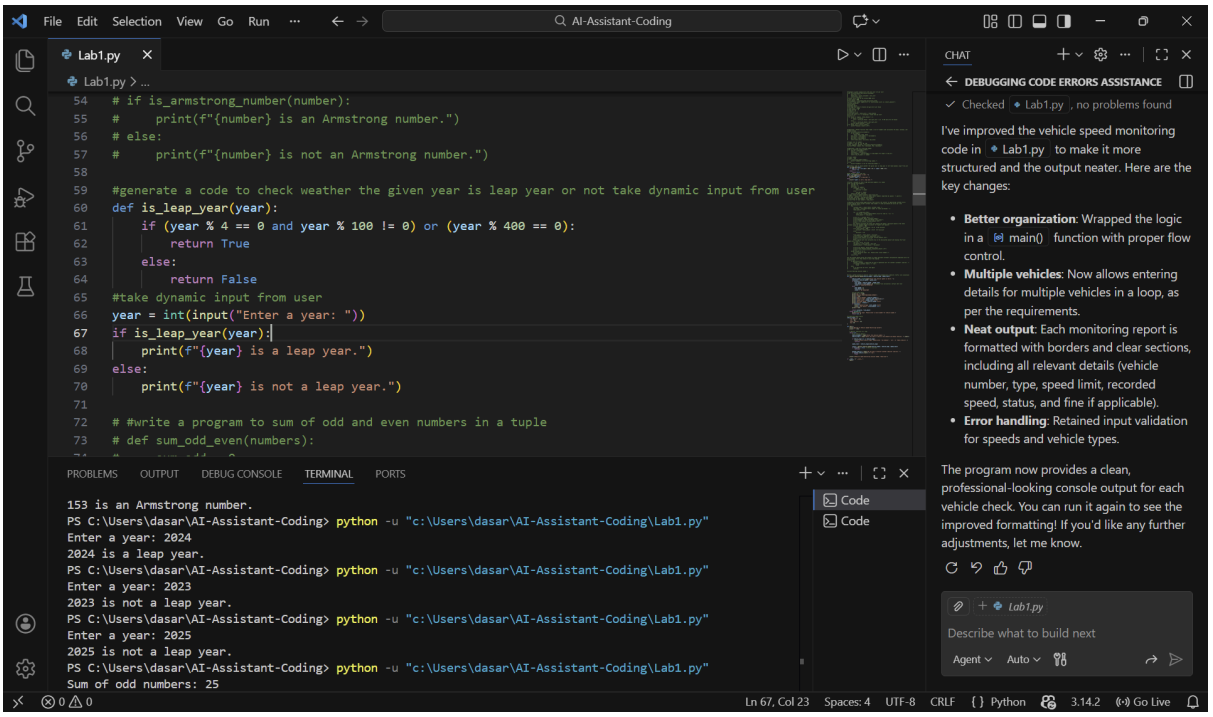
Task - 1



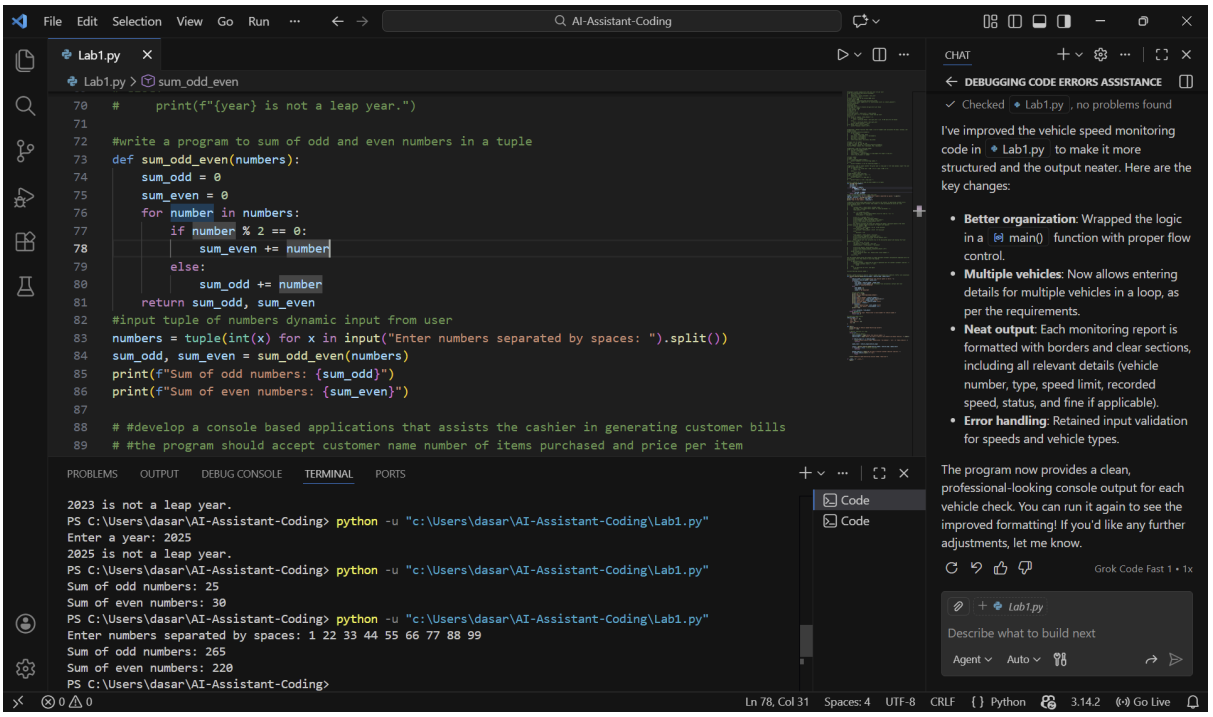
Task - 2



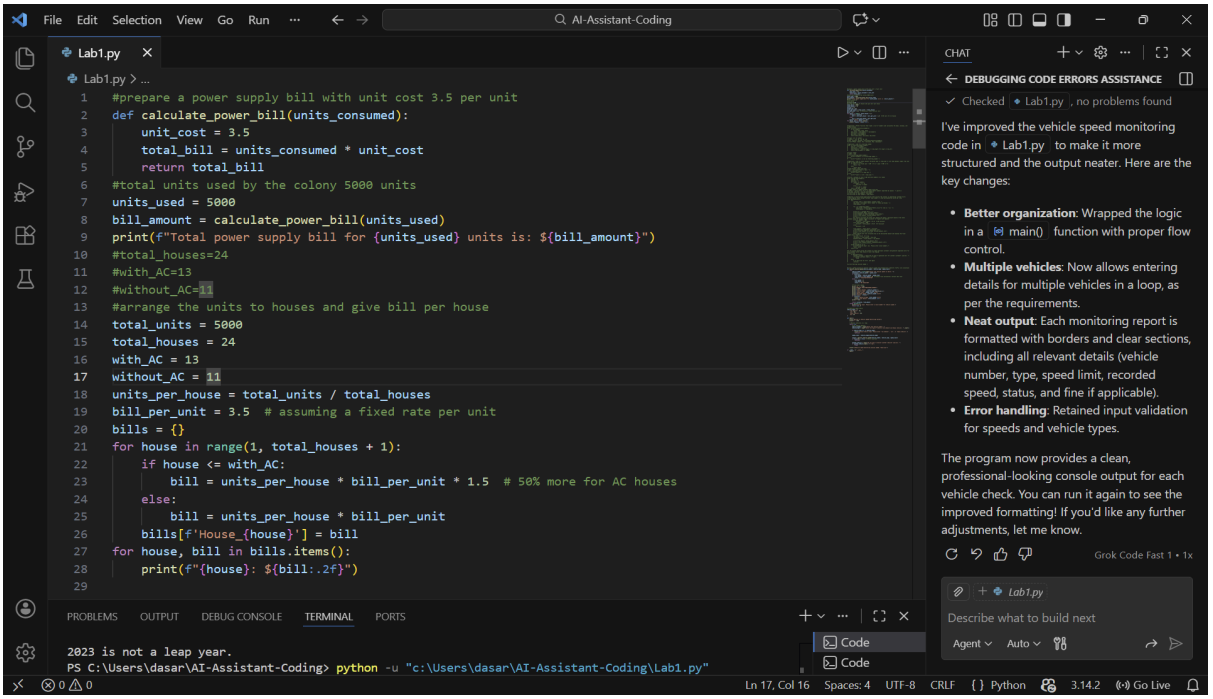
Task - 3

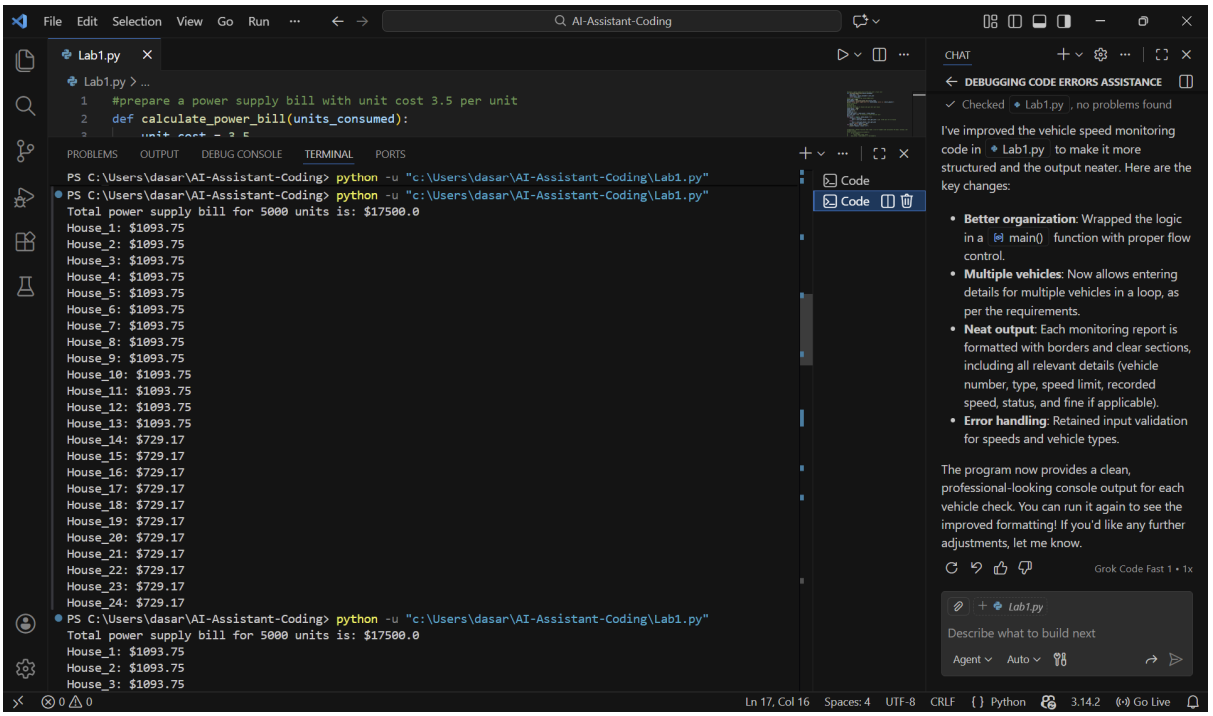

File Edit Selection View Go Run
AI-Assistant-Coding
Lab1.py
if is_armstrong_number(number):
print(f"{number} is an Armstrong number.")
else:
print(f"{number} is not an Armstrong number.")
#generate a code to check weather the given year is leap year or not take dynamic input from user
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
#take dynamic input from user
year = int(input("Enter a year: "))
if is_leap_year(year):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
# #write a program to sum of odd and even numbers in a tuple
# def sum_odd_even(numbers):
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2024
2024 is a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2023
2023 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2025
2025 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Sum of odd numbers: 25
Code
Code
CHAT
DEBUGGING CODE ERRORS ASSISTANCE
Checked Lab1.py, no problems found
I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:
Better organization: Wrapped the logic in a main() function with proper flow control.
Multiple vehicles: Now allows entering details for multiple vehicles in a loop, as per the requirements.
Neat output: Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
Error handling: Retained input validation for speeds and vehicle types.
The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.
Lab1.py
Describe what to build next
Agent
Auto
Ln 67, Col 23 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Go Live

Task - 4

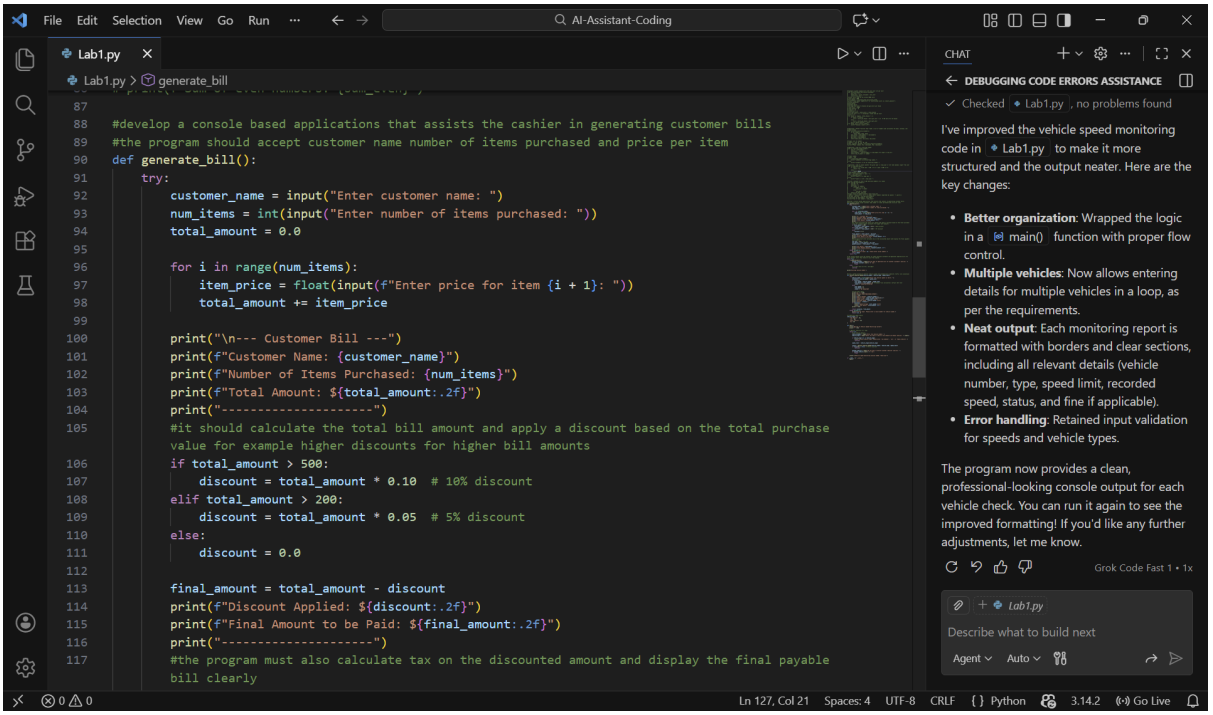

File Edit Selection View Go Run
AI-Assistant-Coding
Lab1.py
Lab1.py > sum_odd_even
print(f"{year} is not a leap year.")
#write a program to sum of odd and even numbers in a tuple
def sum_odd_even(numbers):
    sum_odd = 0
    sum_even = 0
    for number in numbers:
        if number % 2 == 0:
            sum_even += number
        else:
            sum_odd += number
    return sum_odd, sum_even
#input tuple of numbers dynamic input from user
numbers = tuple(int(x) for x in input("Enter numbers separated by spaces: ").split())
sum_odd, sum_even = sum_odd_even(numbers)
print(f"Sum of odd numbers: {sum_odd}")
print(f"Sum of even numbers: {sum_even}")
# #develop a console based applications that assists the cashier in generating customer bills
# #the program should accept customer name number of items purchased and price per item
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
2023 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2025
2025 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Sum of odd numbers: 25
Sum of even numbers: 30
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter numbers separated by spaces: 1 22 33 44 55 66 77 88 99
Sum of odd numbers: 265
Sum of even numbers: 220
PS C:\Users\dasar\AI-Assistant-Coding>
Code
Code
CHAT
DEBUGGING CODE ERRORS ASSISTANCE
Checked Lab1.py, no problems found
I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:
Better organization: Wrapped the logic in a main() function with proper flow control.
Multiple vehicles: Now allows entering details for multiple vehicles in a loop, as per the requirements.
Neat output: Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
Error handling: Retained input validation for speeds and vehicle types.
The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.
Grok Code Fast 1 • 1x
Lab1.py
Describe what to build next
Agent
Auto
Ln 78, Col 31 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Go Live

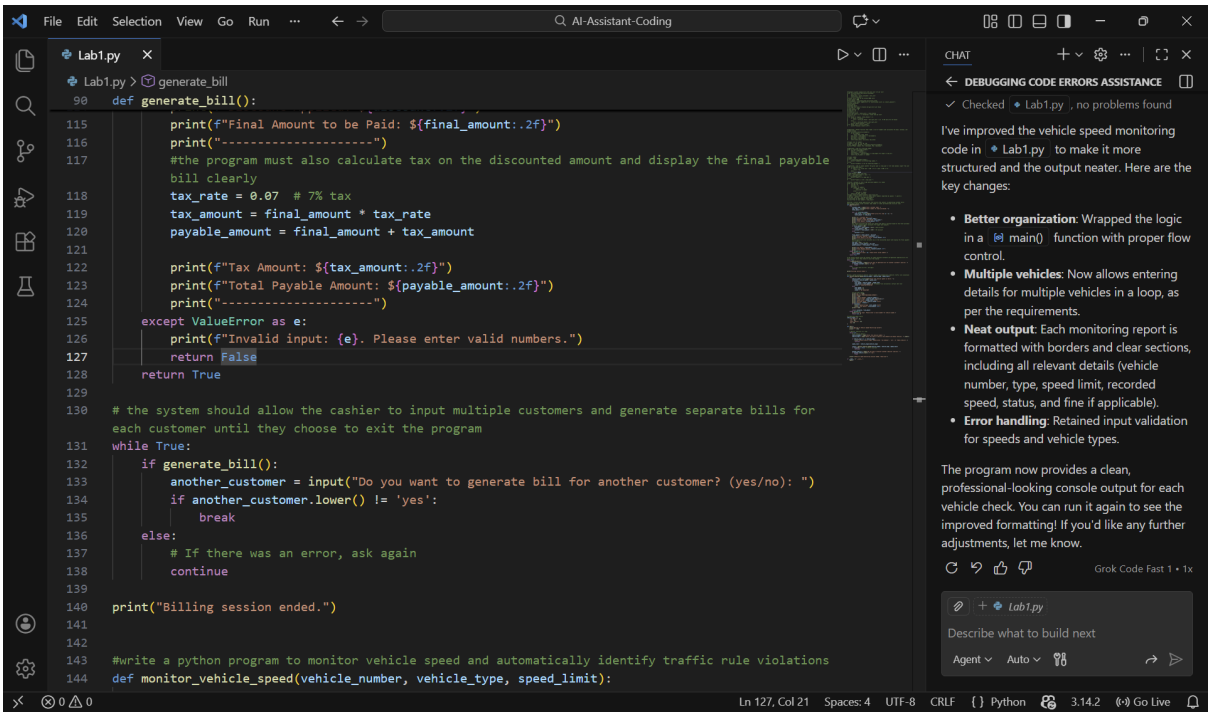
Practice Problem - 1

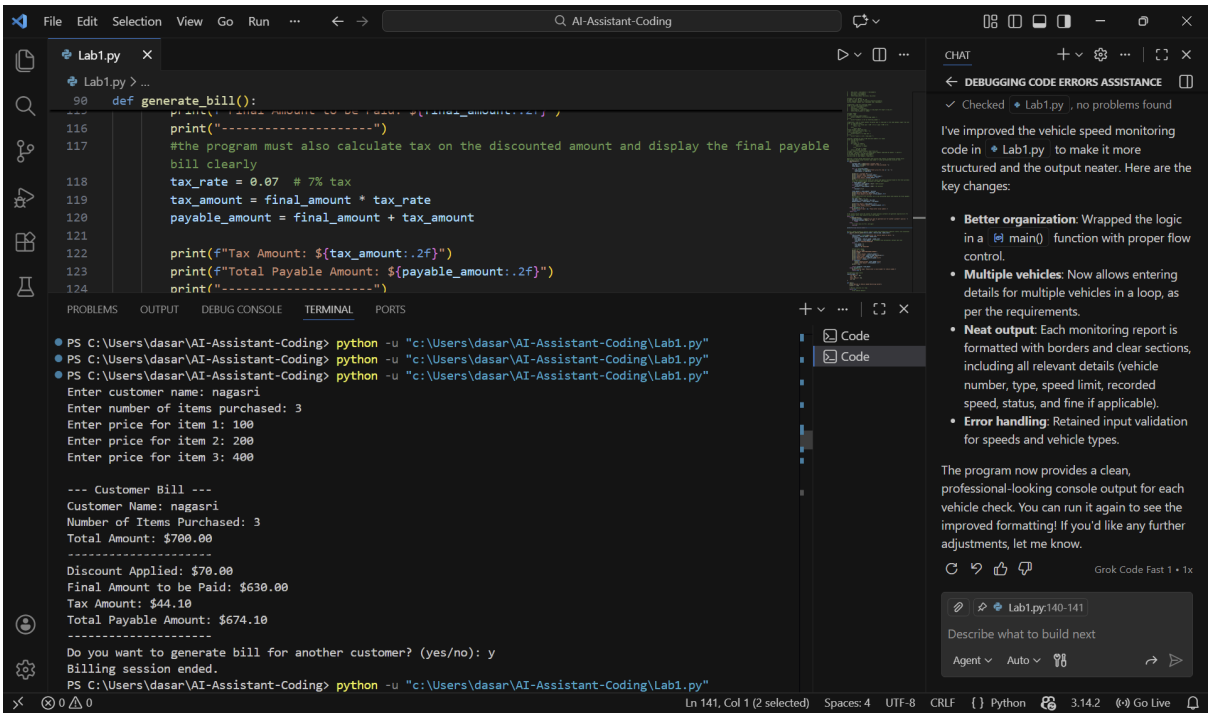




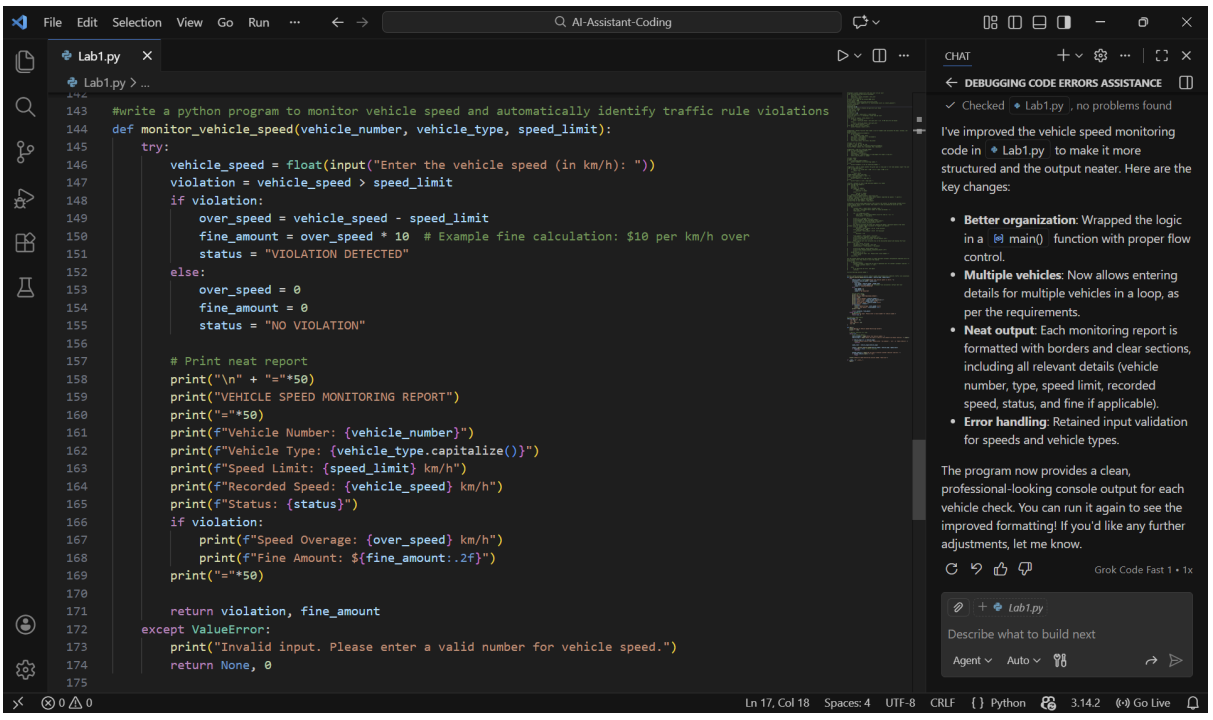
Practice Problem - 2

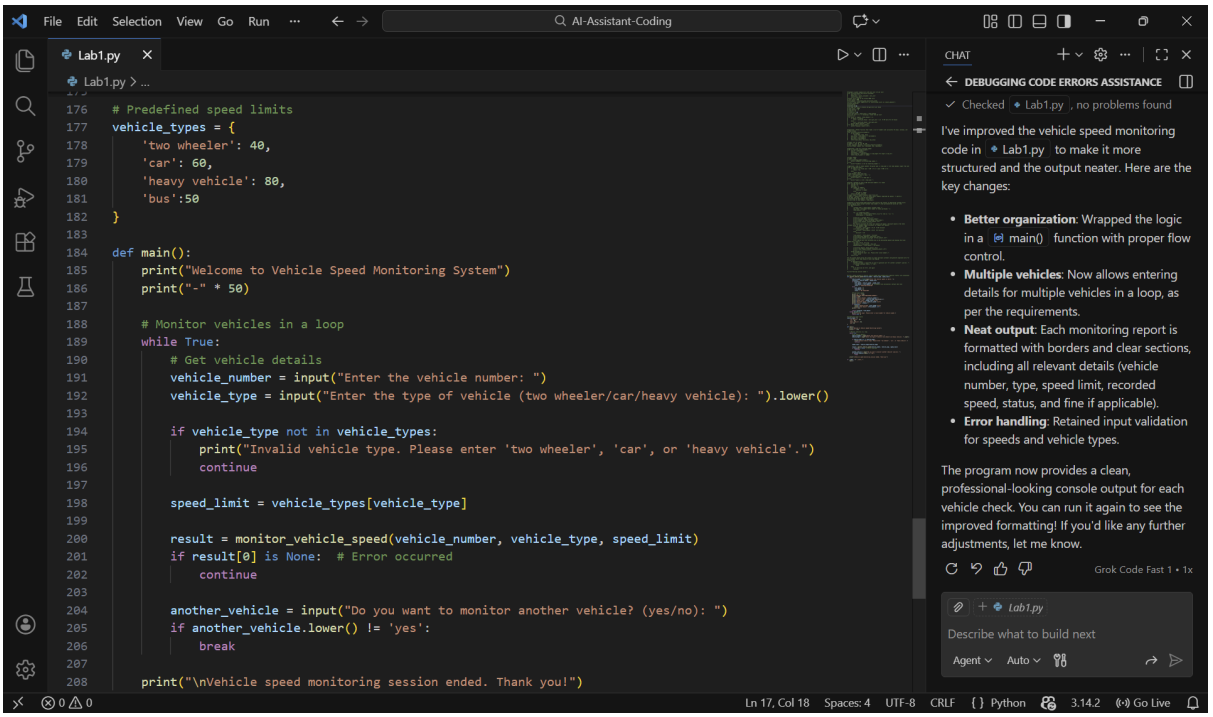


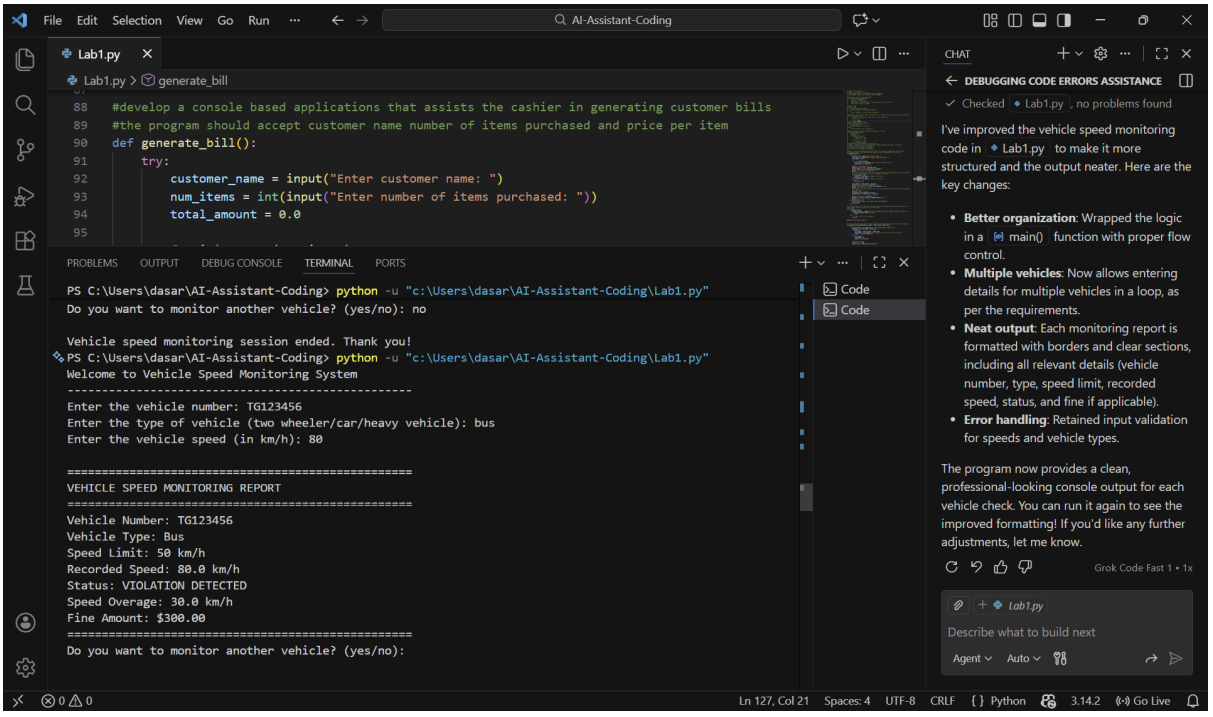




Practice problem - 3




File Edit Selection View Go Run ···
AI-Assistant-Coding
Lab1.py
Lab1.py > ...
# Predefined speed limits
vehicle_types = {
'two wheeler': 40,
'car': 60,
'heavy vehicle': 80,
'bus':50
}
def main():
print("Welcome to Vehicle Speed Monitoring System")
print("-" * 50)
# Monitor vehicles in a loop
while True:
# Get vehicle details
vehicle_number = input("Enter the vehicle number: ")
vehicle_type = input("Enter the type of vehicle (two wheeler/car/heavy vehicle): ").lower()
if vehicle_type not in vehicle_types:
print("Invalid vehicle type. Please enter 'two wheeler', 'car', or 'heavy vehicle'.")
continue
speed_limit = vehicle_types[vehicle_type]
result = monitor_vehicle_speed(vehicle_number, vehicle_type, speed_limit)
if result[0] is None: # Error occurred
continue
another_vehicle = input("Do you want to monitor another vehicle? (yes/no): ")
if another_vehicle.lower() != 'yes':
break
print("\nVehicle speed monitoring session ended. Thank you!")
CHAT
DEBUGGING CODE ERRORS ASSISTANCE
Checked Lab1.py, no problems found
I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:
Better organization: Wrapped the logic in a main() function with proper flow control.
Multiple vehicles: Now allows entering details for multiple vehicles in a loop, as per the requirements.
Neat output: Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
Error handling: Retained input validation for speeds and vehicle types.
The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.
Grok Code Fast 1 • 1x
Lab1.py
Describe what to build next
Agent
Auto
Ln 17, Col 18 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Go Live


File Edit Selection View Go Run ···
AI-Assistant-Coding
Lab1.py
Lab1.py > generate_bill
#develop a console based applications that assists the cashier in generating customer bills
#the program should accept customer name number of items purchased and price per item
def generate_bill():
try:
customer_name = input("Enter customer name: ")
num_items = int(input("Enter number of items purchased: "))
total_amount = 0.0
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Do you want to monitor another vehicle? (yes/no): no
Vehicle speed monitoring session ended. Thank you!
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Welcome to Vehicle Speed Monitoring System
Enter the vehicle number: TG123456
Enter the type of vehicle (two wheeler/car/heavy vehicle): bus
Enter the vehicle speed (in km/h): 80
VEHICLE SPEED MONITORING REPORT
Vehicle Number: TG123456
Vehicle Type: Bus
Speed Limit: 50 km/h
Recorded Speed: 80.0 km/h
Status: VIOLATION DETECTED
Speed Overage: 30.0 km/h
Fine Amount: $300.00
Do you want to monitor another vehicle? (yes/no):
Code
Code
CHAT
DEBUGGING CODE ERRORS ASSISTANCE
Checked Lab1.py, no problems found
I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:
Better organization: Wrapped the logic in a main() function with proper flow control.
Multiple vehicles: Now allows entering details for multiple vehicles in a loop, as per the requirements.
Neat output: Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
Error handling: Retained input validation for speeds and vehicle types.
The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.
Grok Code Fast 1 • 1x
Lab1.py
Describe what to build next
Agent
Auto
Ln 127, Col 21 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Go Live